CSE2246 Assignment 2
Traveling Salesman Problem with Time Windows (TSP-TW)

Due date: June 4th, 2026

The traveling salesman problem (TSP) asks the following question: Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city exactly once and returns to the origin city?

In this assignment, we will focus on a variant of TSP, Traveling Salesman Problem with Time Windows (TSP-TW). In this variant, each city can only be visited within a specified time interval. The salesman may wait before visiting a city if he arrives earlier than the opening time. However, if he arrives after the closing time, that city cannot be visited.

Your goal is to find a valid tour that visits as many cities as possible. Among the solutions that visit the same number of cities, the one with the smaller tour length is considered better.

Inputs are: n cities, with their locations (x and y coordinates) in a 2D grid, and an opening and closing time for each city.

Output is: an ordering (tour) of the cities traveled by the salesman, the number of visited cities, the total tour length, and the completion time of the tour.

The salesman may start from any city. He starts at time 0, travels with unit speed, and must return to the starting city after visiting the selected cities. The starting city is also subject to its own time window. The return to the starting city at the end of the tour is not considered a second visit and does not need to satisfy the starting city time window again.

Distance between two cities is defined as the Euclidean distance rounded to the nearest integer. In other words, you will compute the distance between two cities i and j as follows:

$$d(i,j) = \left\lfloor \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} + 0.5 \right\rfloor$$

If the salesman arrives at a city before its opening time, he may wait until the city opens. Waiting time does not increase the tour length, but it increases the current time. A city i may only be visited if its effective visit time t satisfies open_i <= t <= close_i.

For example, if a city has a time window [20, 50], then an arrival at time 15 is allowed by waiting until time 20, whereas an arrival at time 55 makes the city infeasible for that position in the tour.

Project Specification

Your team (a group of three students) is asked to design and implement a method for finding a solution for the problem that is as close to the optimal solution as possible. Note that the TSP problem is an NP-hard problem, and TSP-TW is at least as hard as the ordinary TSP problem. Given that you must determine the starting city, which cities to visit, and the order of cities to visit while satisfying the time-window constraints, achieving optimal solutions proves exceedingly challenging. Your goal is not to design an algorithm for the optimal solution; rather, you are requested to do your best. This is an open-ended project.

You may do the following:

- Read as much as you want to learn about how to solve the problem. However, you have to cite any resources you use. You may want to start with approximation algorithms and local search heuristics discussed in Chapter 12.
- Use whichever programming language you want.

You may not use the following:

- Existing implementations or subroutines.
- Extensive libraries. If you are not sure whether a library is allowed, check with the instructor.
- Other people's code.

Objective Function

The primary objective is to maximize the number of visited cities. Among all valid solutions that visit the same number of cities, the better solution is the one with the smaller total tour length. In other words, solutions are compared lexicographically:

- More visited cities is always better.
- If two solutions visit the same number of cities, the one with the shorter tour length is better.
- If both the number of visited cities and the tour length are equal, the solution with the smaller completion time may be used as an additional tie-breaker.

Input Format

The input will always be provided as a text file. Each non-empty line describes one city using five space-separated values:

```
city_id x_coordinate y_coordinate open_time close_time
```

The final line of the input file is blank.

Example:

```
0 10 20 0 100
1 25 40 15 60
2 80 12 50 120
```

The following assumptions hold:

- City identifiers are unique integers.
- Coordinates are integers.
- Opening and closing times are integers.
- 0 <= open_time <= close_time.

Output Format

You must output your solution into another text file. The first line should contain three values:

- the number of visited cities, denoted as k,
- the total tour length,
- and the completion time of the tour, including possible waiting times and the final return to the starting city.

The next k lines must list the city IDs in the order they are visited in the tour. The first and last city in the tour are assumed to be connected automatically; therefore, you should not repeat the starting city at the end. The final line of the output file should be left blank.

Example:

```
15 8421 8610
3
7
12
9
...
```

Important Notes

We will provide some sample inputs and a verifying procedure in Google Classroom. This verifying procedure will verify that your output is a VALID output for a given input. It will not verify the optimality of the code, but only its validity.

The verifier will check whether:

- all city IDs in the output are valid,

- no city is visited more than once,
- all time-window constraints are satisfied,
- the reported number of visited cities is correct,
- the reported tour length is correct,
- and the reported completion time is correct.

Using the verifier, you should test that your program reports correct results. Also, note that your code should work in a reasonable time for inputs with up to 50,000 cities.

Test Instances

On June 2nd, we will announce 4 test instances on the website. By June 4th, 23:59, you will be required to submit 4 separate output text files according to the output format, corresponding to each of these test instances.

Project Report

Together with the output text files, you should also submit a project report. The project report should describe the ideas behind your algorithm as completely as possible. It should not exceed 3 pages in length and should use a font size of no less than 10pt.

In the report, you should also:

- explain how your method satisfies the time-window constraints,
- discuss any heuristics, local search methods, or optimization techniques you used,
- cite all resources you used,
- and describe the "division of labor – who did what?".

Grading Policy

60% of your grade will be determined by your project report. Clarity and creativity of your work will significantly affect your grade.

40% of your grade will be determined by your solutions to the test instances. Studies that find better solutions will get higher grades.

Come on to the Contest: "Hodri Meydan!"

For each of the four test inputs, we will identify the best, the second-best, and the third-best performing teams. Best, second, and third projects receive 3 stars, 2 stars, and 1 star, respectively. Therefore, if a project performs best in all four test inputs, it will receive 3 * 4 = 12 stars.

Stars	Bonus Points
1	5
2	7
3	9
4	11
5	13
6	15
7	17
8	19
9	21
10	23
11	25
12	30

Example

Consider the following input file:

```
0 0 0 0 100
1 3 4 8 20
2 6 0 10 30
```

There are three cities. The corresponding coordinates and time windows are:

City	Coordinates	Time window
0	(0,0)	[0,100]
1	(3,4)	[8,20]
2	(6,0)	[10,30]

Suppose the salesman visits the cities in the following order:

$0 \rightarrow 1 \rightarrow 2 \rightarrow 0$

The distances are computed as follows:

d(0,1)=floor(sqrt((0-3)^2+(0-4)^2)+0.5)=floor(5+0.5)=5

d(1,2)=floor(sqrt((3-6)^2+(4-0)^2)+0.5)=floor(5+0.5)=5

d(2,0)=floor(sqrt((6-0)^2+(0-0)^2)+0.5)=floor(6+0.5)=6

The salesman starts at city 0 at time t=0.

City 0 has time window [0,100], so the visit is valid.

The salesman then travels from city 0 to city 1: t = 0 + 5 = 5

City 1 has time window [8,20]. Since the salesman arrives early, he waits until time 8.

The salesman then travels from city 1 to city 2: t = 8 + 5 = 13

City 2 has time window [10,30], so the visit is valid.

Finally, the salesman returns from city 2 to city 0: t = 13 + 6 = 19

The total tour length is: 5 + 5 + 6 = 16

The total completion time is: 19

Therefore, one valid output file is:

```
3 16 19
0
1
2
```
